

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

AIM :

Implement the A* Search algorithm for solving a pathfinding problem.

Implement the Recursive Best-First Search algorithm for the same problem.

Compare the performance and effectiveness of both algorithms.

CODE: A* SEARCH

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = {

    'Tata Power Residential Colony': {
        'Raghuleela Mega Mall': 2.4,
        'Thakur College': 2.8
    },

    'Raghuleela Mega Mall': {
        'Charkop': 2.2,
        'Malad-Marve Road': 3.1
    },

    'Thakur College': {
        'Malad-Marve Road': 3.4
    },

    'Charkop': {
        'Vishals Magic Cricket Academy': 2.3
    },

    'Malad-Marve Road': {
        'Vishals Magic Cricket Academy': 2.0
    },

    'Vishals Magic Cricket Academy': {
        'Children Park and Gymnasium': 0.03
    },

    'Children Park and Gymnasium': {}
}
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
heuristics = {

    'Tata Power Residential Colony': 5.56,
    'Raghuleela Mega Mall': 4.05,
    'Thakur College': 4.60,
    'Charkop': 2.55,
    'Malad-Marve Road': 2.10,
    'Vishals Magic Cricket Academy': 0.025,
    'Children Park and Gymnasium': 0
}

node_positions = {

    'Tata Power Residential Colony': (4,8),
    'Raghuleela Mega Mall': (2,6),
    'Thakur College': (6,6),
    'Charkop': (2,4),
    'Malad-Marve Road': (6,4),
    'Vishals Magic Cricket Academy': (4,2),
    'Children Park and Gymnasium': (4,0)
}

def a_star_search(graph, heuristics, start, goal):

    priority_queue = []
    heapq.heappush(priority_queue, (heuristics[start], start, [start], 0))

    visited = set()

    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
visited.add(current)

if current == goal:
    return path, g_score

for neighbor, edge_weight in graph[current].items():

    if neighbor not in visited:

        new_g = g_score + edge_weight
        new_f = new_g + heuristics[neighbor]

        heapq.heappush(
            priority_queue,
            (new_f, neighbor, path + [neighbor], new_g)
        )

return None, float("inf")

start = "Tata Power Residential Colony"
goal = "Children Park and Gymnasium"

optimal_path, total_distance = a_star_search(
    graph,
    heuristics,
    start,
    goal
)

print("Optimal Path:")
print(" -> ".join(optimal_path))

print("\nTotal Road Distance:", round(total_distance,2), "km")

G = nx.DiGraph()

for node, neighbors in graph.items():
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
for neighbor, weight in neighbors.items():
    G.add_edge(node, neighbor, weight=weight)

plt.figure(figsize=(12,8))

path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2600,
    node_color="lightblue"
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=2,
    edge_color="gray",
    arrows=True
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=4,
    edge_color="orange",
    arrows=True
)

labels = {
    node: f"{node} \nh(n)={heuristics[node]}"
    for node in G.nodes()
}

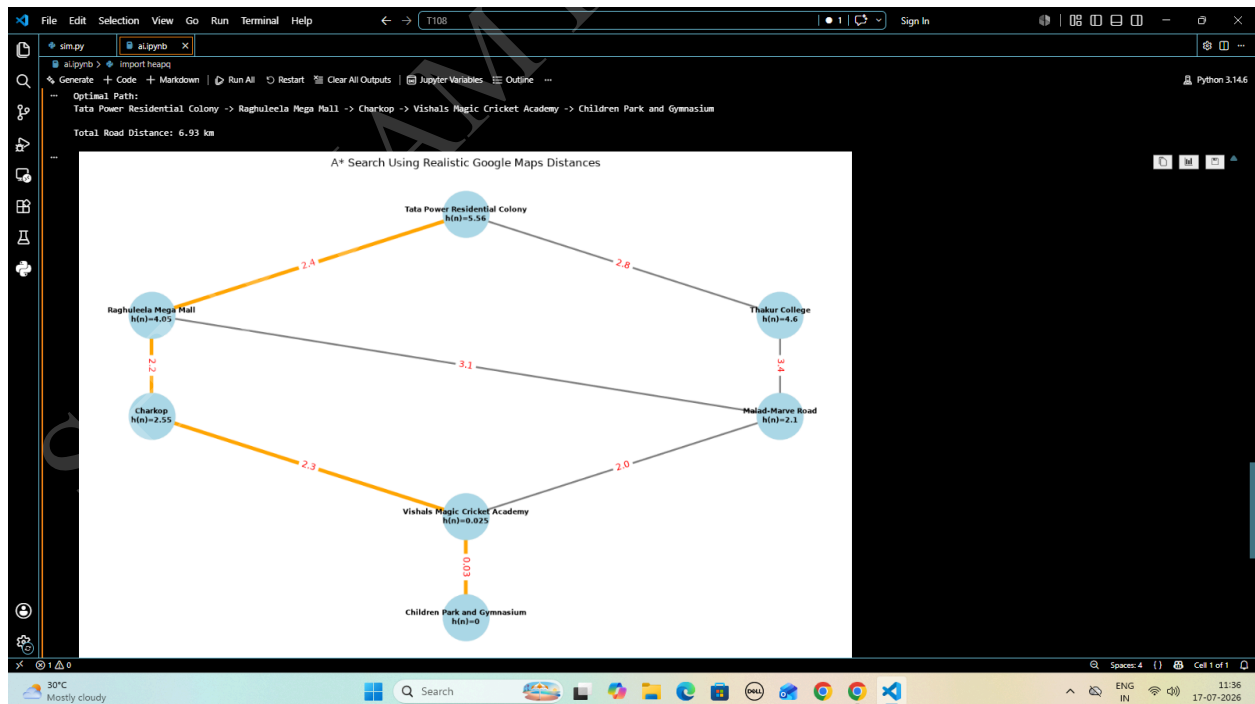
nx.draw_networkx_labels(
    G,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
node_positions,  
labels=labels,  
font_size=8,  
font_weight="bold"  
)  
  
edge_labels = nx.get_edge_attributes(G, "weight")  
  
nx.draw_networkx_edge_labels(  
    G,  
    node_positions,  
    edge_labels=edge_labels,  
    font_color="red"  
)  
  
plt.title("A* Search Using Realistic Google Maps Distances")  
  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

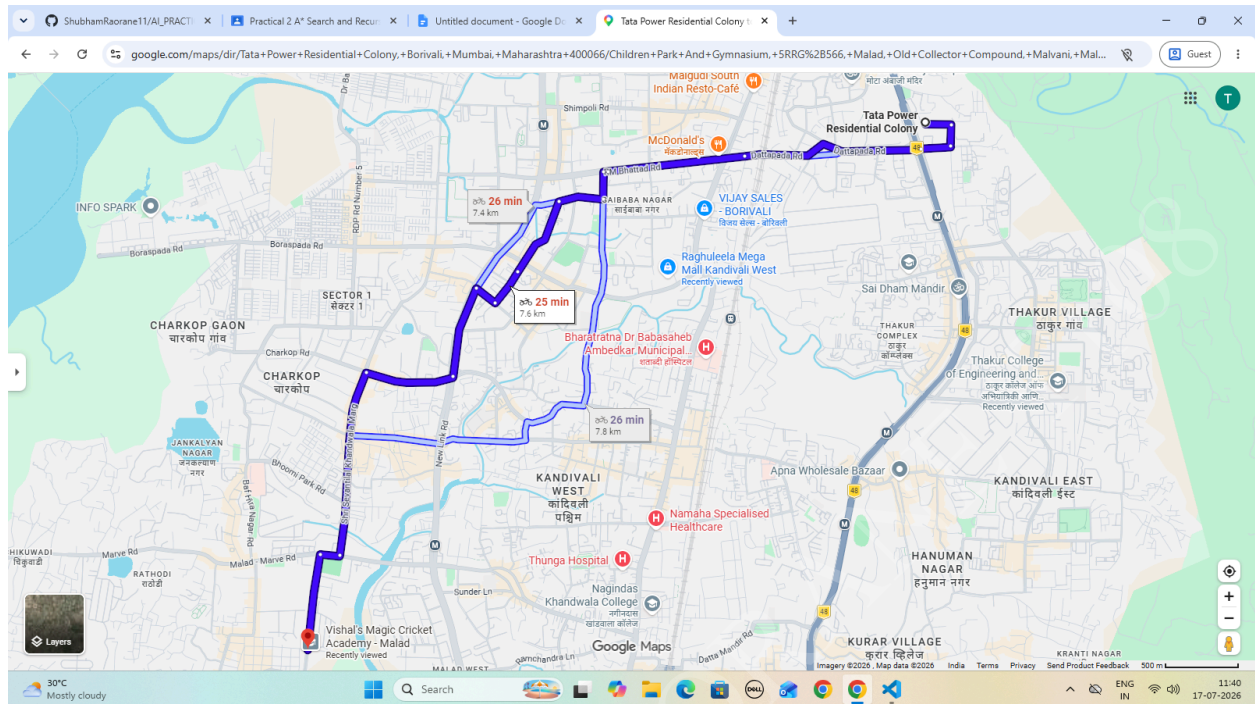
OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

MAP:



CODE: `import heapq`

`import matplotlib.pyplot as plt`

`import networkx as nx`

`graph = {`

```
'Tata Power Residential Colony': {
    'Raghuleela Mega Mall': 2.4,
    'Thakur College': 2.8
},
```

```
'Raghuleela Mega Mall': {
    'Charkop': 2.2,
    'Malad-Marve Road': 3.1
},
```

```
'Thakur College': {
    'Malad-Marve Road': 3.4
```

SHUBHAM RAORANE T108

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
},  
  
  'Charkop': {  
    'Vishals Magic Cricket Academy': 2.3  
  },  
  
  'Malad-Marve Road': {  
    'Vishals Magic Cricket Academy': 2.0  
  },  
  
  'Vishals Magic Cricket Academy': {  
    'Children Park and Gymnasium': 0.03  
  },  
  
  'Children Park and Gymnasium': {}  
}
```

```
heuristics = {  
  
  'Tata Power Residential Colony': 5.56,  
  'Raghuleela Mega Mall': 4.05,  
  'Thakur College': 4.60,  
  'Charkop': 2.55,  
  'Malad-Marve Road': 2.10,  
  'Vishals Magic Cricket Academy': 0.025,  
  'Children Park and Gymnasium': 0  
}
```

```
node_positions = {  
  
  'Tata Power Residential Colony': (4,8),  
  'Raghuleela Mega Mall': (2,6),  
  'Thakur College': (6,6),  
  'Charkop': (2,4),  
  'Malad-Marve Road': (6,4),  
  'Vishals Magic Cricket Academy': (4,2),  
}
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
'Children Park and Gymnasium': (4,0)

}

# Recursive A* Function

def recursive_a_star(graph, heuristics, goal, priority_queue, visited):

    if not priority_queue:
        return None, float("inf")

    f_score, current, path, g_score = heapq.heappop(priority_queue)

    if current in visited:
        return recursive_a_star(
            graph,
            heuristics,
            goal,
            priority_queue,
            visited
        )

    visited.add(current)

    # Goal reached
    if current == goal:
        return path, g_score

    # Expand neighbors
    for neighbor, distance in graph[current].items():

        if neighbor not in visited:
```


SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
new_g = g_score + distance

new_f = new_g + heuristics[neighbor]

heapq.heappush(
    priority_queue,
    (
        new_f,
        neighbor,
        path + [neighbor],
        new_g
    )
)

# Recursive call
return recursive_a_star(
    graph,
    heuristics,
    goal,
    priority_queue,
    visited
)

# Starting Node and Goal

start = "Tata Power Residential Colony"

goal = "Children Park and Gymnasium"

pq = []

heapq.heappush(
    pq,
    (
        heuristics[start],
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
        start,
        [start],
        0
    )
)

visited = set()

optimal_path, total_distance = recursive_a_star(
    graph,
    heuristics,
    goal,
    pq,
    visited
)

print("Optimal Path:")
print(" -> ".join(optimal_path))

print("\nTotal Road Distance:",
      round(total_distance,2),
      "km")

# Graph Visualization

G = nx.DiGraph()

for node, neighbors in graph.items():

    for neighbor, weight in neighbors.items():

        G.add_edge(node, neighbor, weight=weight)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
plt.figure(figsize=(12,8))

path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [
    edge for edge in G.edges()
    if edge not in path_edges
]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2600,
    node_color="lightblue"
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=2,
    edge_color="gray",
    arrows=True
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=4,
    edge_color="orange",
    arrows=True
)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
labels = {

    node:
    f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()

}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=labels,
    font_size=8,
    font_weight="bold"
)

edge_labels = nx.get_edge_attributes(
    G,
    "weight"
)

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red"
)

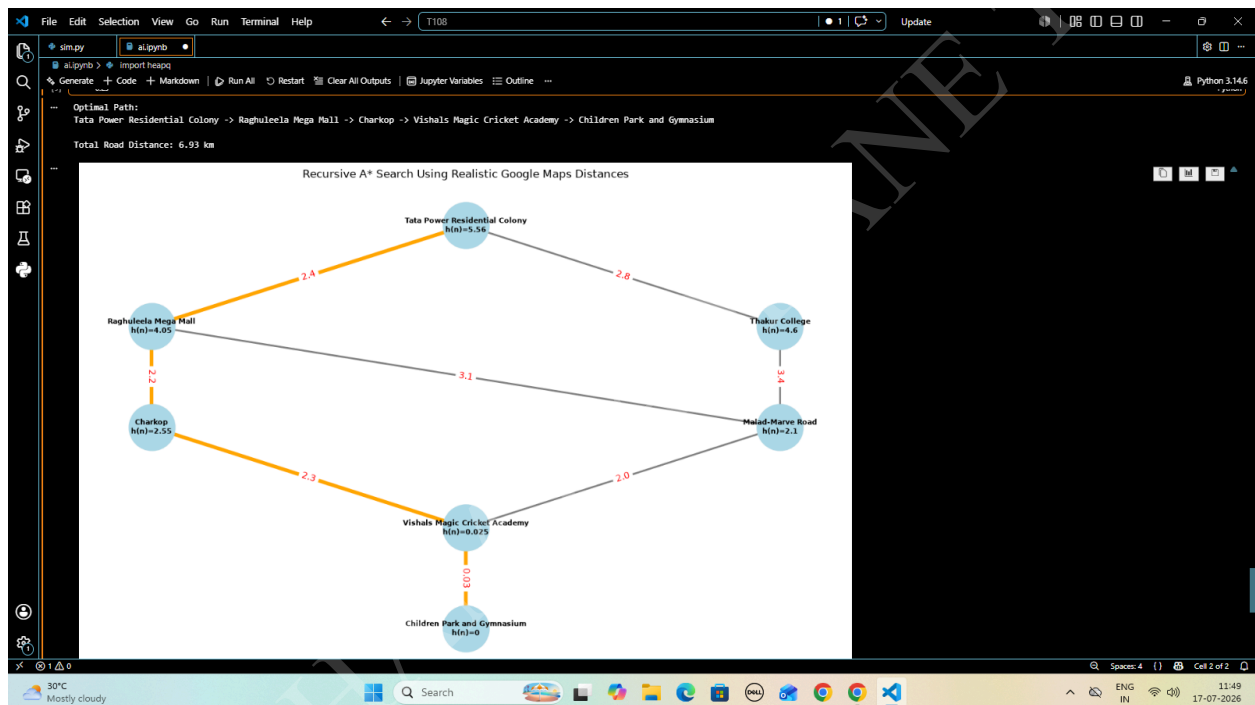
plt.title(
    "Recursive A* Search Using Realistic Google Maps Distances"
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
)  
  
plt.axis("off")  
  
plt.tight_layout()  
  
plt.show()
```

OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

RECURSIVE INPUT

```
import matplotlib.pyplot as plt

#
-----
# 1. NETWORK DEFINITION (Mahim -> Airoli)
#
-----
# Road distances g(n) from a node to its connected neighbors
graph = {
    'Mahim': {
        'Bandra': 3.5,
        'Sion': 4.2
    },

    # Route 1 & 3 branches via Bandra / West
    'Bandra': {
        'Powai': 12.0
    },

    'Powai': {
        'Vikhroli': 4.0,    # Continues Route 1
        'Mulund': 6.2      # Continues Route 3
    },

    # Route 2 branch via Central Arteries
    'Sion': {
        'Chembur': 5.5
    },

    'Chembur': {
        'Vikhroli': 7.5
    },

    # Final entries into Airoli
    'Vikhroli': {
        'Airoli': 8.0      # Merges Route 1 & Route 2
    },

    'Mulund': {
        'Airoli': 6.5      # Route 3 approach
    },
}
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
'Airoli': {}  
}  
  
# Straight-line distance heuristics h(n) targeting Airoli (Goal)  
heuristics = {  
    'Mahim': 20,  
    'Bandra': 18,  
    'Sion': 16,  
    'Powai': 10,  
    'Chembur': 12,  
    'Vikhroli': 7,  
    'Mulund': 5,  
    'Airoli': 0  
}  
  
# 2D coordinates (X, Y) scaled for map plotting layout  
coords = {  
    'Mahim': (1, 2),  
    'Bandra': (1, 4),  
    'Sion': (3, 2),  
    'Chembur': (4.5, 3),  
    'Powai': (2.5, 6),  
    'Vikhroli': (5, 6),  
    'Mulund': (4, 8),  
    'Airoli': (7, 7)  
}  
  
#  
-----  
# 2. RECURSIVE BEST-FIRST SEARCH (RBFS) IMPLEMENTATION  
#  
-----  
  
def rbfs_search(start, goal):  
    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'),  
    path=[start])  
    return path, cost  
  
def rbfs(node, goal, g, f_limit, path):  
    if node == goal:
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
    return True, path, g, g

neighbors = graph[node]
if not neighbors:
    return False, [], 0, float('inf')

successors = []
for neighbor, distance in neighbors.items():
    if neighbor not in path: # Prevent cyclic looping
        next_g = g + distance
        next_f = max(next_g + heuristics[neighbor], g +
heuristics[node])
        successors.append([next_f, neighbor, next_g])

if not successors:
    return False, [], 0, float('inf')

while True:
    # Sort successors by their current evaluation score f(n)
    successors.sort(key=lambda x: x[0])
    best = successors[0]

    if best[0] > f_limit:
        return False, [], 0, best[0]

    alternative_f = successors[1][0] if len(successors) > 1 else
float('inf')

    # Recurse downward into the best branch
    success, result_path, total_g, returned_f = rbfs(
        best[1], goal, best[2], min(f_limit, alternative_f), path +
[best[1]]
    )

    best[0] = returned_f

    if success:
        return True, result_path, total_g, returned_f
```


SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
#
-----

# RUN SEARCH
#
-----

path, total_dist = rbfs_search('Mahim', 'Airoli')
print(f"Optimal RBFS Path: {' ' -> ' '.join(path)} ({round(total_dist, 1)} km)")

#
-----

# 3. GRAPH PLOTTING
#
-----

plt.figure(figsize=(10, 8))

# Draw Road Links
for node, neighbors in graph.items():
    x1, y1 = coords[node]
    for neighbor, dist in neighbors.items():
        x2, y2 = coords[neighbor]

        # Check if this edge belongs to the calculated path
        is_path = node in path and neighbor in path and
path.index(neighbor) == path.index(node) + 1
        color, width = ('#2ecc71', 3.5) if is_path else ('#bdc3c7', 1.5)

        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width,
zorder=1)

        plt.text((x1 + x2) / 2, (y1 + y2) / 2, f"{dist}km", color='red',
fontsize=9, ha='center', va='bottom')

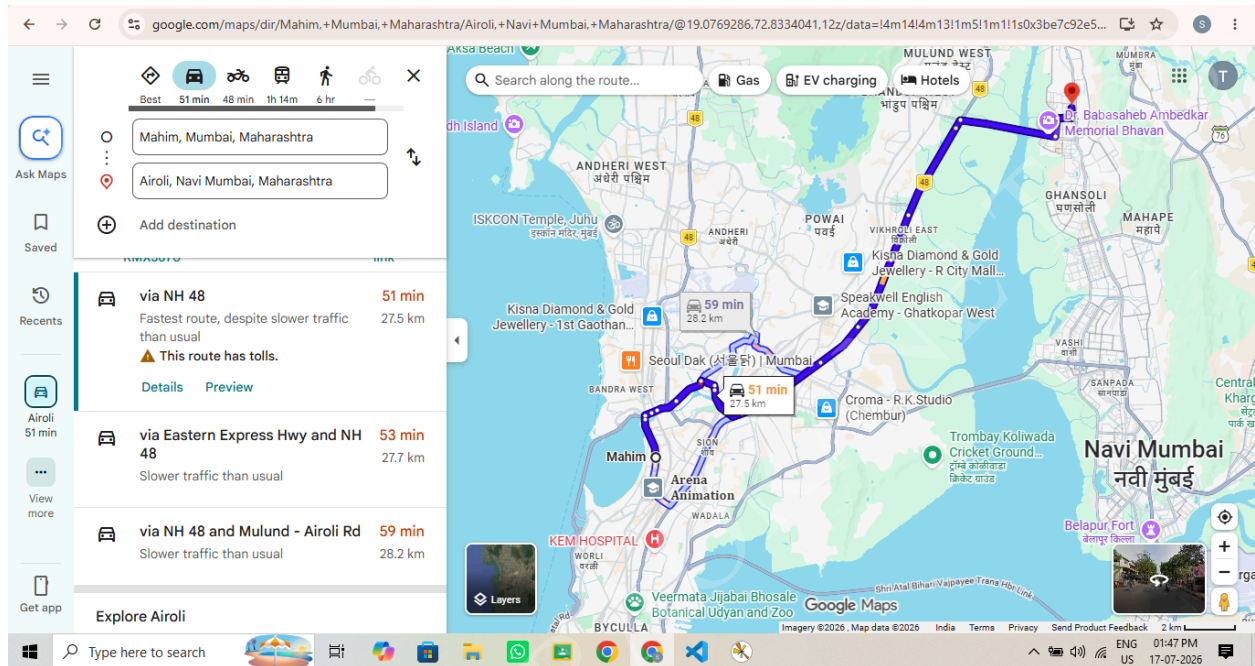
# Draw Nodes
for node, (x, y) in coords.items():
    color = '#f1c40f' if node in path else '#3498db'
    plt.scatter(x, y, color=color, s=1200, edgecolors='black', zorder=2)
    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center',
va='center', color='white', fontsize=8, fontweight='bold')
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

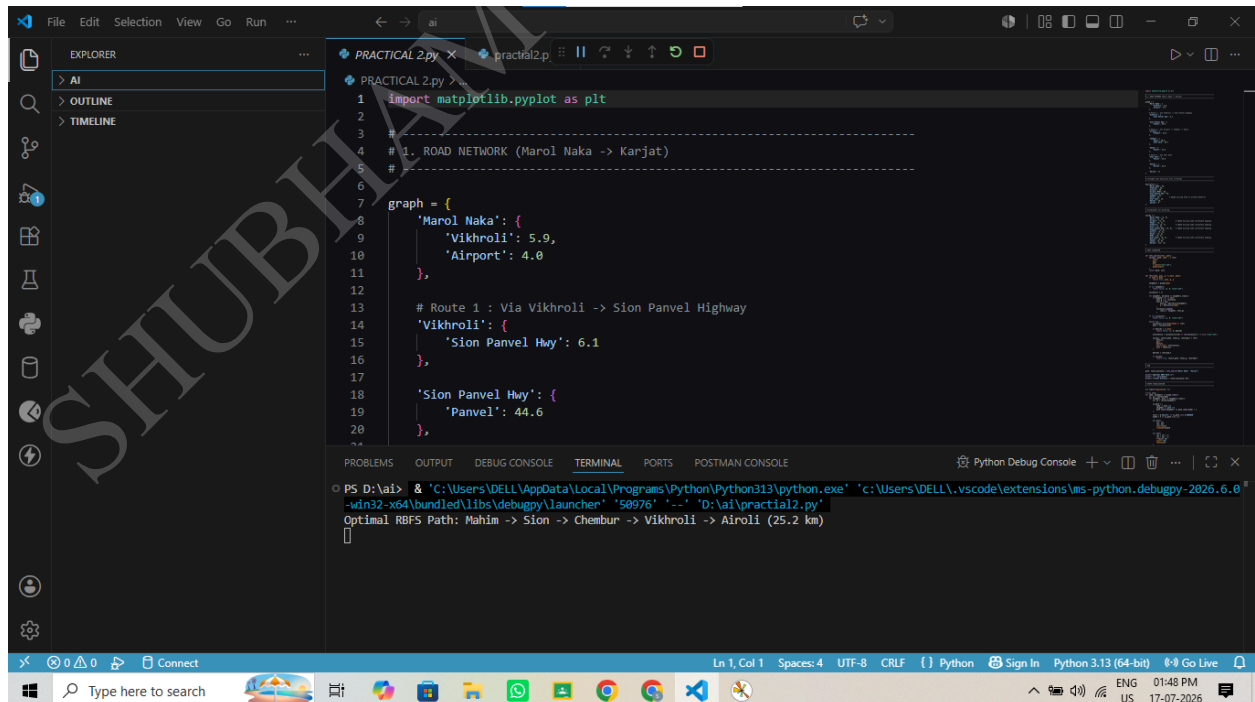
SUBJECT : AI

```
plt.title(f"RBFS Shortest Route Map: Mahim → Airoli\nTotal Optimal  
Distance: {round(total_dist, 1)}km", fontsize=12, fontweight='bold')  
plt.axis('off')  
plt.show()
```

MAP



OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Figure 1

